# Finance Text-to-SQL: Technical Guide

Repository investigation, interview reference, and operational notes
Repository state inspected: 20 October 2026.
Evidence labels: Implemented means directly present in the checked-in code or current repository artifacts; Documented means stated in repository documentation; Inference means a conclusion from implementation details; Proposed means future work.

## 1. Executive summary

Implemented: This repository is a prototype pipeline that fine-tunes a causal language model to translate questions about a small personal-finance relational schema into SQL. Training is supervised fine-tuning (SFT) over instruction/input/output JSON examples. src/utils/model_loader.py can load the configured TinyLlama model in 4-bit NF4 when CUDA and bitsandbytes appear available. src/training/lora_config.py attaches LoRA adapters. A FastAPI service generates SQL; a separate evaluator executes predictions against SQLite and compares them with reference-query results.

Intended users (inference): Developers experimenting with domain adaptation, and consumers asking finance questions in natural language. No user study or product deployment evidence is present.

System boundary: The service returns generated SQL and a schema string. The API does not connect to a live finance database or execute the returned SQL. The local evaluator executes SQL against a synthetic SQLite fixture. No authentication, authorization, SQL execution sandbox, hosted deployment configuration, CI workflow, or automated test suite is present in the tracked file inventory.

Status: This is best described as an educational/prototyping project. Production readiness, measured model quality, and operational performance are not verified from the repository.

## 2. Repository map and evidence

```
  |   Area   |   Main files   |   Responsibility   |
  |  ---  |  ---  |  ---  |
  |   Configuration   |   [`configs/training_config.yaml`](../configs/training_config.yaml)   |   Base model,
input/output paths, hyperparameters, schema text   |
  |   Data generation   |   [`data/gen_dataset.py`](../data/gen_dataset.py)   |   Build template-based question/SQL
examples   |
  |   SQLite fixture   |   [`data/setup_test_db.py`](../data/setup_test_db.py)   |   Create six-table synthetic
test database   |
  |   Training   |   [`src/training/train.py`](../src/training/train.py),
[`src/training/formatting.py`](../src/training/formatting.py),
[`src/training/data_split.py`](../src/training/data_split.py),
[`src/training/lora_config.py`](../src/training/lora_config.py)   |   Split, format, load base model, attach
adapter, run TRL SFT   |
  |   Model loading   |   [`src/utils/model_loader.py`](../src/utils/model_loader.py)   |   Tokenizer and causal LM
loading with quantized/CUDA/CPU branches   |
  |   Inference helper   |   [`src/inference/gen_sql.py`](../src/inference/gen_sql.py)   |   Python generation
helper   |
  |   HTTP API   |   [`api/main.py`](../api/main.py)   |   FastAPI schema, health, and SQL generation endpoints   |
  |   Batch inference   |   [`scripts/generate_predictions.py`](../scripts/generate_predictions.py)   |   Generate
predictions for the deterministic held-out split   |
  |   Evaluation   |   [`src/eval/eval.py`](../src/eval/eval.py),
```

[`src/eval/eval_accuracy.py`](../src/eval/eval_accuracy.py) | Exact string match, SQLite result comparison, error labels |
| Orchestration | [`scripts/run_pipeline.py`](../scripts/run_pipeline.py), [`scripts/validate_setup.py`](../scripts/validate_setup.py) | Sequential pipeline and setup checks |
| Packaging | [`requirements.txt`](../requirements.txt), [`docker/Dockerfile`](../docker/Dockerfile), [`docker/docker-compose.yml`](../docker/docker-compose.yml) | Python dependencies and a proposed containerized API |

Tracked files include no CI workflow, migration system, separate production database configuration, or automated tests. models/ has local adapter artifacts in this checkout, but .gitignore excludes model outputs; the tracked repository does not include the trained adapter weights. Generated data artifacts are also ignored. Local presence does not establish reproducible availability to a fresh clone.

# 3. Problem and data contract

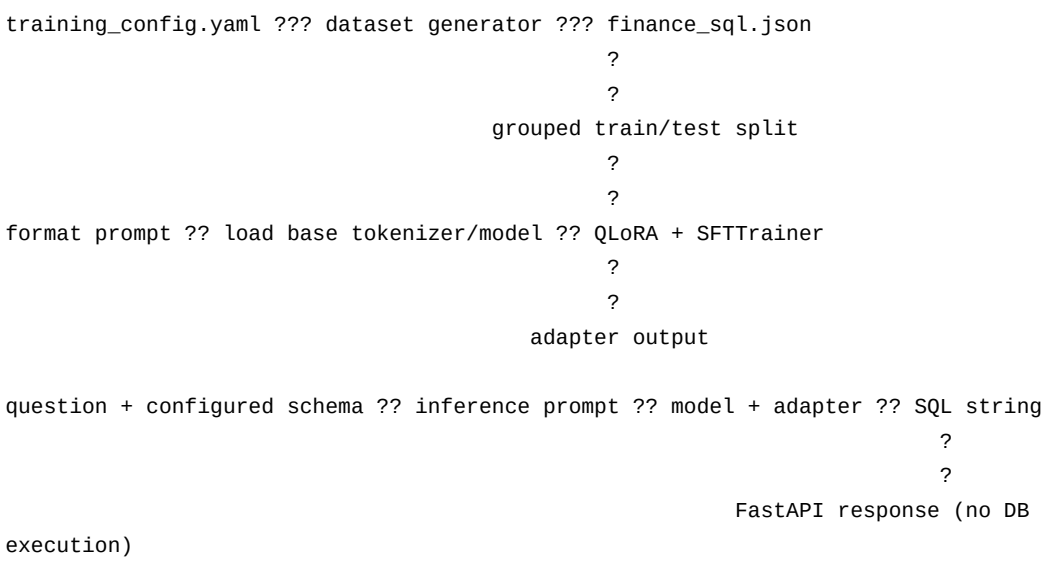
The model input is a natural-language question accompanied by a schema description. The training record has instruction, input, and output fields. The formatter turns each record into a simple <|user|> ... <|assistant|> text sequence. The target is SQL text; there is no structured SQL AST, tool call, or execution result in the model output contract.

The configured base is TinyLlama/TinyLlama-1.1B-Chat-v1.0; the output directory is named models/phi3-sql, which is a potentially confusing historical label and does not match the configured base. The configured schema covers users, accounts, expenses, income, budgets, and transactions, with user/account foreign-key relationships described in the test DB setup.

data/gen_dataset.py creates examples from hand-written templates and random choices of names, categories, thresholds, and limits. It covers aggregates, grouping, filters, joins, subqueries, ordering, budget comparisons, and CASE expressions. It is synthetic template data, not evidence of real customer data or broad natural-language coverage. The generator shuffles without setting a seed. Some prompts recur and some templates have the same question for every generated row.

The test database also uses random values and the current date for its date fields. It does not seed its random generator. A future regeneration can therefore change the fixture and recent-date query results. Query behavior for an empty sum or an absent category follows SQLite semantics; callers should not assume a non-null numeric answer.

# 4. Architecture and execution flow



```
held-out samples ?? batch generation ?? predictions.json ?? SQLite evaluator
```

## Training path

1. `src/training/train.py` loads YAML from a working-directory-relative path and seeds Transformers/PyTorch through `set_seed`.
2. Hugging Face Datasets loads the JSON file. `grouped_split_indices` groups rows by the entire `input` string, shuffles distinct prompts using a local seeded RNG, and assigns whole groups to the test set until a target row count is reached. This prevents identical input strings from straddling train/test, but does not group semantically equivalent paraphrases. Group sizes can make the test split exceed the target; at least one row is selected for test.
3. The formatter adds instruction, input, and expected SQL into the sequence consumed by `SFTTrainer`.
4. `load_model` loads tokenizer and base model. With CUDA and the bitsandbytes import/attribute check, the model uses 4-bit NF4 quantization and automatic device mapping. Otherwise it chooses CUDA FP16 or CPU FP32.
5. Training enables gradient checkpointing, disables the model KV cache during training, attaches rank-16 LoRA modules to `q_proj` and `v_proj`, and runs TRL `SFTTrainer` with configured epochs, batch size, gradient accumulation, learning rate, sequence length, an 8-bit paged AdamW optimizer, cosine schedule, and epoch evaluation/checkpointing.
6. `trainer.save_model` writes the final adapter to the configured output directory. The `test` split here is used as the trainer evaluation set; it is not a separately described untouched final benchmark.

## Inference paths

There are three implementations, not one shared serving path: src/inference/gen_sql.py, scripts/generate_predictions.py, and api/main.py each build their own prompt and call generate. The Python helper and API decode the entire generated sequence and split on the assistant marker; batch inference slices off the prompt token IDs and decodes only new tokens. Generation parameters differ. The batch path uses greedy decoding (do_sample=False); the API explicitly samples (do_sample=True) with temperature 0.2. The API does not set an explicit random seed per request, so repeated responses can vary.

The API loads configuration at module import and loads the base model plus adapter at application startup. /health returns whether the model object exists; /schema returns configured schema text; /generate-sql accepts { "question": "..." } and returns sql and schema_used. The API uses .to("cuda") for tokenized input even though its startup can select devices automatically; CPU-only serving is therefore not supported by that path as written. torch.float16 is passed when loading its model regardless of device. Startup/model download failures are not translated into a custom health state.

## Evaluation path

scripts/generate_predictions.py uses the grouped split and writes data/processed/predictions.json. src/eval/eval.py expects each record to contain predicted and output, then runs batch metrics against a SQLite file. Exact match lowercases, collapses whitespace, and removes trailing semicolons. Execution accuracy compares sets of returned rows, not ordered sequences or multisets. Thus row order and duplicate multiplicity do not affect this metric. It checks outputs on one fixture and cannot establish general semantic correctness.

Error classification attempts prediction and reference execution and distinguishes missing columns/tables, syntax-like operational errors, other execution errors, and unequal result sets. The declared SEMANTIC label is not returned in the current classifier, and schema-column/table lists are collected but not used for further validation. A prediction that fails execution is marked not correct. The evaluator executes generated SQL directly against

its SQLite connection without a read-only authorizer, statement allowlist, or timeout.

# 5. Design decisions and trade-offs

| Choice | Evidence and rationale | Trade-off / interview answer |
| --- | --- | --- |
| SFT on a causal LM | `SFTTrainer` learns next-token prediction over prompt plus SQL target. | Straightforward for text generation; model can emit malformed, unsafe, or incorrect SQL. |
| LoRA adapters | Rank 16, alpha 32, dropout 0.05; target query/value projection modules. | Reduces trainable parameter count and adapter storage versus full fine-tuning. Targets only these modules and rank were chosen in code; no repository ablation establishes they are optimal. |
| 4-bit NF4 quantization | `BitsAndBytesConfig` branch, with bfloat16 if supported else float16 compute. | Reduces base-weight memory, at cost of compatibility constraints and possible numerical effects. Fallback to full precision may exceed available hardware. |
| Gradient checkpointing | Enabled in training and SFT config. | Saves activation memory by recomputing activations during backward pass, increasing computation. |
| Paged 8-bit AdamW | Configured as `paged_adamw_8bit`. | Optimizer state memory is reduced; requires compatible bitsandbytes setup. |
| SQLite fixture evaluation | Lightweight local execution against a known schema. | Reproducible design intent but fixture is random/date-dependent; set-based results mask order and duplicates and may reward coincidental equal results. |
| FastAPI | Typed Pydantic request/response and simple HTTP interface. | Simple serving surface. No authentication, throttling, request validation beyond string type, SQL policy enforcement, or concurrency/load design is present. |

Alternatives such as full fine-tuning, larger/richer datasets, constrained decoding, schema linking, SQL parser validation, database-aware prompting, or execution-guided repair are proposed directions, not implemented features.

# 6. Configuration, setup, and operations

Dependencies in requirements.txt are unpinned names, so a fresh install is not version-locked. The model and dataset require external model hub access on first use; actual access/cache behavior depends on environment. No .env.example or required environment variables were found in the tracked file list. .env is ignored; secret contents were not inspected or included.

Documented setup begins with Python 3.10+, a virtual environment, and pip install -r requirements.txt; training is launched with python -m src.training.train. Data and test DB generation are separate commands. The repository also has scripts/validate_setup.py and scripts/run_pipeline.py; the latter runs generation, DB creation, training, prediction, and evaluation sequentially and exits on the first failure.

Configuration paths are relative to the current working directory, so invoke commands from repository root. The Dockerfile uses WORKDIR /app, copies requirements and then the repository, and starts Uvicorn on port 8000. Compose declares build: . in docker/docker-compose.yml; Compose resolves this context relative to the Compose file, so its context is docker/. The Dockerfile is in that directory, but requirements.txt is in the repository root and is unavailable to COPY requirements.txt . from that context. A likely correction is build: { context: .., dockerfile: docker/Dockerfile } (not applied). Also, the image installs unpinned dependencies and copies the local repository, including whatever files are in the build context after .dockerignore filtering. These are repository-level configuration observations, not runtime test results.

The README's stated VRAM and training-time figures are documented claims only; there are no reproducible benchmark artifacts or environment details proving those figures in this checkout. Training hardware, observed runtime, and achieved metrics are not verified from the repository.

The current code uses report_to="none"; although it calls mlflow.set_experiment, no explicit MLflow run or metric logging is visible in the training script. Do not describe this as verified MLflow metric tracking without inspecting a successful run.

# 7. Security, reliability, and correctness concerns

1. **Generated SQL is not execution-safe by default.** The API currently returns SQL only, but downstream execution must apply authorization, read-only connections, statement restrictions, query limits/timeouts, and tenant scoping. The evaluation code itself runs arbitrary predicted SQL on the fixture connection.
2. **No authentication or authorization.** The API exposes generation and schema endpoints with no access control in the implementation.
3. **Device mismatch.** API inputs are always moved to CUDA; CPU fallback in the shared loader does not make API inference CPU-compatible.
4. **Serving and training generation differ.** Stochastic API decoding, greedy batch decoding, and different output slicing make results and output cleanup non-equivalent.
5. **Metric limitations.** Set equality discards order and duplicates. Exact match is a lexical proxy, not semantic validation. One small changing SQLite fixture is insufficient as a generalization benchmark.
6. **Potential SQL/data semantic mismatch.** Some generated question wording and SQL templates deserve review, e.g. a “transactions in category” question queries `expenses`, while `transactions` has no category column. A quarter template subtracts two months from the start of the current month, which is not a calendar-quarter calculation. Such defects can make labels wrong even when a model reproduces them.
7. **Synthetic data leakage and diversity.** The split groups identical `input` strings, but many examples share fixed templates or are duplicated by random choice. This can make a row-level held-out score optimistic and does not show generalization to unseen phrasing/schema.
8. **Non-deterministic fixture/data generation.** Dataset generation and SQLite sample data use random values without seeding; SQLite rows use the current date. Training has a seed, but reproducibility begins only after generated inputs are fixed.
9. **Schema/config duplication.** `schema` and `schema_simple` are both maintained in YAML, and the test DB is maintained separately in Python. They can drift.
10. **Deployment config mismatch.** Compose does not identify the Dockerfile in its nested directory. The Dockerfile's default installation has no pinned or GPU-specific dependency selection.

# 8. Reproduction and troubleshooting

From repository root, the documented flow is:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python data/gen_dataset.py
python data/setup_test_db.py
python scripts/validate_setup.py
python -m src.training.train
python -m scripts.generate_predictions
python -m src.eval.eval --dataset data/processed/predictions.json --db
data/processed/finance.db --output data/processed/eval_results.json
```

The validation script can attempt model tokenizer access/download; it does not run full training. Model hub/network access, sufficient disk, compatible CUDA/PyTorch/bitsandbytes builds, and enough RAM/VRAM are practical requirements. Windows bitsandbytes compatibility is

explicitly listed by the README as a known issue. If quantized loading fails or its availability check falls back, inspect the branch and memory requirements before attributing the issue to the data pipeline. If generation works in batch but not API on a CPU host, inspect the API's hard-coded CUDA transfer. If Compose cannot locate its Dockerfile, set the Dockerfile path in the Compose build definition or run a build that points to docker/Dockerfile (proposed operational fix; not applied here).

# 9. Interview-ready explanation

### 30-second overview

“This is a prototype for translating personal-finance questions into SQL. It uses a small instruction-tuned causal language model and supervised fine-tuning with a LoRA adapter, with an optional 4-bit NF4 base-model load to reduce training memory. A FastAPI endpoint returns generated SQL using a configured schema. Separately, a batch pipeline evaluates predictions by normalized exact match and by executing both predicted and reference SQL on a synthetic SQLite database. The implementation is a useful end-to-end experiment, but its data and evaluator are small, and the API does not safely execute generated SQL or establish production readiness.”

### Likely follow-up questions

Why LoRA/QLoRA-> The implementation configures 4-bit NF4 loading and rank-16 LoRA on query/value projections to reduce memory and update only adapter parameters. This is a plausible small-hardware adaptation strategy; no controlled comparison in the repository proves it beats full fine-tuning or another rank/target-module choice.

How do you avoid duplicate prompt leakage-> The split groups exactly identical input strings before selecting the test partition. It does not deduplicate semantically equivalent questions or template families, and a prompt group can push test size above its target.

How do you measure correctness-> The evaluator has normalized string exact match and SQLite result-set equality. They answer different questions: textual similarity versus equal outputs on one database. The latter ignores order/duplicate count and is not a proof of semantic equivalence.

How is the system deployed-> A FastAPI application and Docker artifacts exist. The API loads model and adapter on startup and offers health/schema/generation routes. The Compose file's build path needs correction, and the code lacks production controls such as authentication, safe SQL execution, CPU-compatible device handling, monitoring, and resource limits.

What would you improve first-> Make data generation and fixtures deterministic; audit every question/SQL pair; strengthen evaluation with multiple fixtures, ordered/multiset-aware comparisons where appropriate, and parse/safety checks; unify prompt construction; add device-aware serving; constrain generated SQL to authorized read-only operations; pin dependencies; fix and verify container build; then report held-out results with environment and sample counts.

# 10. Evidence gaps and verification status

? Test suite/results: no tracked automated tests found; no tests were run for this investigation.
? Accuracy, latency, throughput, cost, memory consumption, scaling, and production uptime: **Not verified from the repository.**
? Dataset source beyond the synthetic template generator and any external real-world data: **Not verified from the repository.**
? Deployment environment and live database integration: **Not verified from the repository.**
? API behavior on CPU-only hosts and a fresh Docker Compose build: identified as

code/configuration concerns; not executed here.
? Current local adapter files and generated datasets may exist in a developer checkout but are ignored by Git and are not portable evidence of a reproducible release.
? No application code or existing files were modified for this report. Only this guide and its PDF deliverable are added.

# Appendix A: source file index

See the repository map in section 2. Core implementation references are configs/training_config.yaml, data/gen_dataset.py, data/setup_test_db.py, src/training/{train.py,formatting.py,data_split.py,lora_config.py}, src/utils/model_loader.py, src/inference/gen_sql.py, api/main.py, scripts/generate_predictions.py, and src/eval/{eval.py,eval_accuracy.py}.